

Agente autónomo de ciberseguridad basado en aprendizaje por refuerzo

Javier Rivero Iglesias

Abril 2026

Índice general

1. Introducción	2
2. Problema y alcance	3
3. Evolución del repositorio	4
3.1. Etapa histórica: NSL-KDD + DQN	4
3.2. Migración a CICIDS2017 + esquema canónico + QRDQN	4
4. Línea base actual	6
4.1. Invariantes	6
4.2. Recompensa	6
5. Resultados reproducibles	7
5.1. Entrenamiento CICIDS2017 + QRDQN	7
5.2. Validación	7
6. Fase 2 y cambio de distribución	9
7. Conclusiones y siguientes pasos	10
8. Fuentes internas del repositorio	11

Capítulo 1

Introducción

Esta memoria de trabajo resume la evolución real del repositorio TFG_CYBER_AI. El objetivo del proyecto es estudiar si un agente de aprendizaje por refuerzo puede tomar decisiones binarias sobre tráfico de red:

- 0 = PERMIT
- 1 = BLOCK

La historia del TFG tiene dos etapas claras. La primera fue una fase de validación conceptual sobre NSL-KDD, útil para comprobar que el problema podía formularse como una política binaria con recompensas sensibles al coste. La segunda, que es la línea actualmente mantenida, migro a CICIDS2017, fijo un esquema canónico de 76 variables de flujo y adopto QRDQN como algoritmo principal.

La memoria deja de presentar NSL-KDD como base final del sistema. Desde el punto de vista del repositorio, la línea principal es ahora CICIDS2017 + QRDQN + esquema canónico, mientras que NSL-KDD queda como antecedente histórico.

Capítulo 2

Problema y alcance

El trabajo se divide en dos fases:

- F1. Fase 1: entrenamiento y validacion offline.** Se entrena un agente sobre datasets etiquetados y se mide su rendimiento con artefactos reproducibles.
- F2. Fase 2: inferencia offline en laboratorio.** Se aplican el modelo y el preprocesado aprendidos a flujos extraídos en un laboratorio privado para medir el efecto del cambio de distribución respecto al dataset.

No forma parte de la línea base actual el bloqueo activo en producción. La Fase 2 sigue siendo de inferencia y análisis, no de despliegue defensivo autónomo.

Capítulo 3

Evolución del repositorio

3.1. Etapa histórica: NSL-KDD + DQN

La primera rama importante del proyecto utilizó NSL-KDD y agentes DQN junto a un baseline supervisado Random Forest. Su función fue:

- validar la formulación PERMIT/BLOCK
- explorar funciones de recompensa
- comparar RL frente a un baseline clásico

Los resultados históricos archivados muestran que Random Forest superaba ligeramente a los primeros DQN:

Experimento	Modelo	Accuracy	Recall ataque
E01	DQN	0.7602	0.6000
E02	Random Forest	0.7693	0.6150
E05	DQN	0.7563	0.5955

Esta etapa fue útil como prueba de concepto, pero no quedó alineada con la futura Fase 2 ni con una representación moderna de flujos.

3.2. Migración a CICIDS2017 + esquema canónico + QRDQN

La línea mantenida introdujo tres decisiones estructurales:

- usar CICIDS2017 como dataset principal
- congelar un esquema canónico de 76 variables de flujo
- añadir una máscara de ausencia/presencia para construir observaciones de 152 dimensiones

La semántica de la máscara quedó fijada como:

- 1 = present / valid
- 0 = missing / imputed

Sobre esa representación se consolidaron:

- src/canonical_schema.py
- src/load_cicids2017.py
- src/train_rl_defender.py
- src/validate_checks.py
- src/validate_leave_one_csv_out.py
- scripts/predict_real_traffic_v2.py

Capítulo 4

Línea base actual

4.1. Invariantes

La implementación actual mantiene los siguientes invariantes:

- `FEATURES_CANON` contiene 76 características
- la observación final siempre tiene 152 dimensiones
- las etiquetas son 0 = BENIGN y 1 = ATTACK
- no se deben usar IPs, timestamps absolutos, Flow IDs ni proxies directos de puertos

4.2. Recompensa

El código actual está estandarizado en torno a:

```
reward_config = {tp = 1.5, fp = -1.5, fn = -5.0, omission = 0.0}
```

No obstante, varios artefactos históricos comprometidos fueron generados con otros valores. El mejor run histórico de CICIDS2017, por ejemplo, uso `fp = -2.0`. Por eso es necesario distinguir siempre entre *defaults actuales* y *configuración histórica del run*.

Capítulo 5

Resultados reproducibles

5.1. Entrenamiento CICIDS2017 + QRDQN

Los principales runs comprometidos en el repositorio son:

ID	Filas	Pasos	Split	Accuracy	Recall atk	F1 atk	Recompensa
C01 smoke	50k	5k	random	0.9697	0.99958	0.96922	1.5, −1.0, −5.0, 0.0
C01 full	250k	100k	random	0.99618	0.99980	0.99628	1.5, −1.0, −5.0, 0.0
C02 fast	100k	10k	random	0.9766	0.99959	0.98123	1.5, −1.0, −5.0, 0.0
C03 full	500k	100k	random	0.99859	0.99945	0.99876	1.5, −2.0, −5.0, 0.0

El mejor artefacto histórico comprometido es C03_qrdqn_cicids2017_canonical_full_random_202 con:

- accuracy: 0.99859
- precisión ataque: 0.99806
- recall ataque: 0.99945
- F1 ataque: 0.99876

5.2. Validación

La rama CICIDS2017 gana credibilidad cuando se pasó de medir solo *random split* a incorporar validaciones más exigentes:

Check	Artefacto	Resultado clave	Lectura
A	VAL_checks_A_20260212_235443	accuracy 0.9939	evaluacion directa fuerte
B	VAL_checks_B_20260212_235736	shuffled 0.4773	sin fuga aparente
C	VAL_checks_C_20260213_004847	accuracy 0.84135	generalizacion mas dura

El Check C es especialmente importante porque entrena en los grupos **Monday**, **Tuesday** y **Wednesday**, y evalúa sobre **Thursday** y **Friday**. En ese régimen, el recall de ataque cae a 0.52954, lo que deja claro que la generalización entre días sigue siendo uno de los principales retos del proyecto.

La validación leave-one-exact-CSV-out ya existe en código, pero no tiene todavía un artefacto agregado comprometido bajo `runs/validation/`. Por tanto, su metodología puede describirse, pero no sus métricas finales.

Capítulo 6

Fase 2 y cambio de distribución

La entrada mantenida para Fase 2 es `scripts/predict_real_traffic_v2.py`. Esta tubería reutiliza el modelo, el escalador y los percentiles del mejor run de CICIDS2017 para inferir sobre flujos obtenidos en el laboratorio.

Los artefactos comprometidos muestran dos comportamientos muy distintos sobre tráfico benigno:

Run	CSV	Block rate	Allow rate	z abs mean
P2v2_pred_20260224_004121	flows_benign.csv	1.0	0.0	1.11489
P2v2_pred_20260408_230318	flows_benign.csv	0.0	1.0	0.68709

La conclusión operativa es que la Fase 2 sigue siendo sensible al cambio de distribución entre CICIDS2017 y tráfico real del laboratorio. Por eso cualquier afirmación sobre comportamiento en Fase 2 debe ir siempre ligada al `RUN_ID` exacto y no formularse como una propiedad estable del sistema.

Capítulo 7

Conclusiones y siguientes pasos

El repositorio ya no es solo una colección de experimentos iniciales sobre NSL-KDD. La línea mantenida basada en CICIDS2017, esquema canónico y QRDQN constituye una base mucho mas sólida, con artefactos reproducibles y validaciones mejor definidas.

Sin embargo, permanecen tres riesgos principales:

- la brecha entre *random split* y generalización realista entre días
- la ausencia de un artefacto leave-one-exact-CSV-out comprometido
- la sensibilidad de la Fase 2 al cambio de distribución

Los siguientes pasos razonables del TFG son:

- conservar un run reproducible de leave-one-exact-CSV-out
- repetir inferencia de Fase 2 sobre tráfico benigno y mixto con trazabilidad completa
- decidir si hace falta calibración adicional con datos del laboratorio
- mantener la memoria sincronizada con `docs/results.md` y los artefactos reales

Capítulo 8

Fuentes internas del repositorio

Esta memoria debe apoyarse en los siguientes documentos y artefactos internos:

- README.md
- .github/AGENT_CONTEXT.md
- docs/results.md
- experiments/nslkdd_experiments.md
- experiments/cicids2017_qrdqn_experiments.md
- runs/cicids2017/C03_qrdqn_cicids2017_canonical_full_random_20260223_232439/
- runs/validation/VAL_checks_A_20260212_235443/
- runs/validation/VAL_checks_B_20260212_235736/
- runs/validation/VAL_checks_C_20260213_004847/
- runs/phase2/P2v2_pred_20260224_004121/
- runs/phase2/P2v2_pred_20260408_230318/